🎨 Frontend Architecture - Complete Components Guide

Master All Frontend Components & Structure

This comprehensive guide explains every component, service, guard, interceptor, model, and module in your frontend: what they do, their responsibilities, how they connect, and what role each plays in the Angular application.

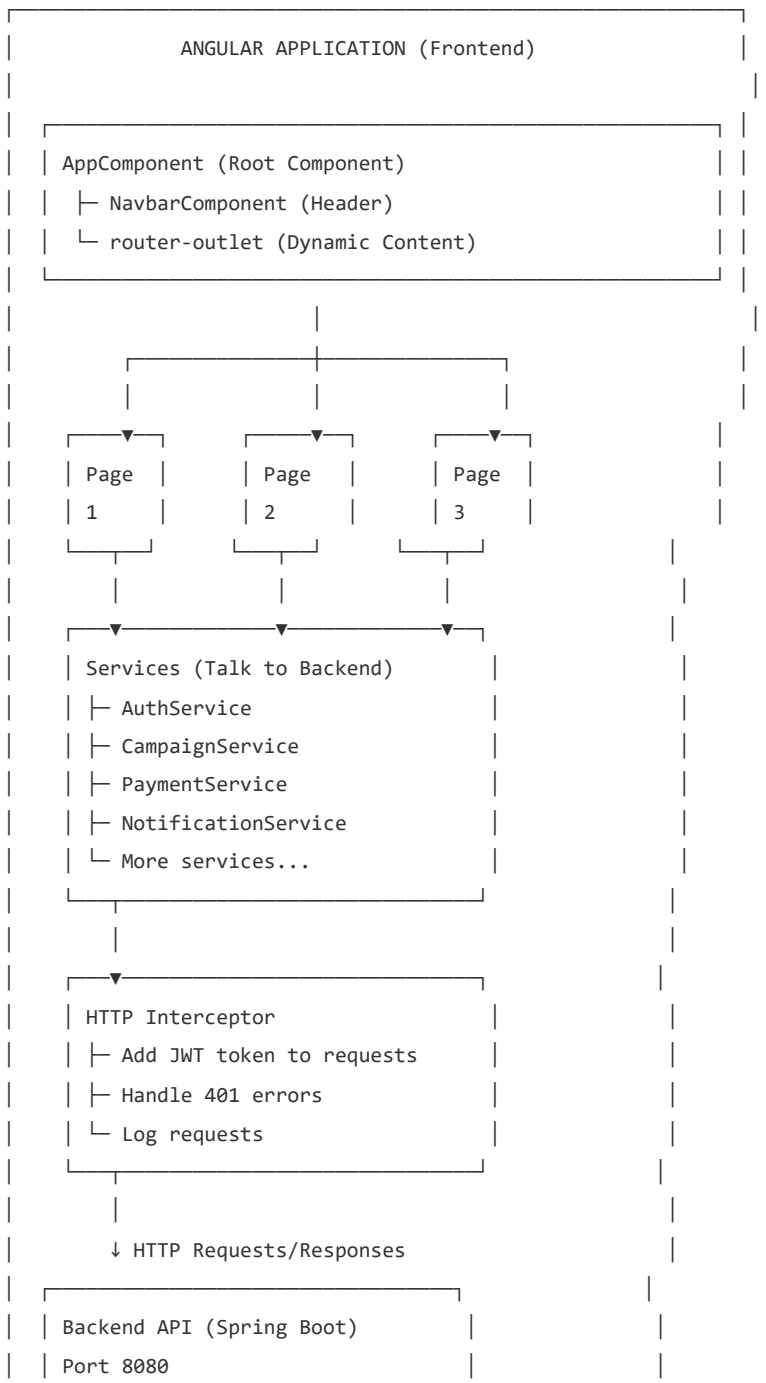
📑 Complete Table of Contents

14. Complete User Flows

1 Frontend Architecture Overview

What is Angular?

Angular is a frontend framework by Google that builds interactive web applications. Your app uses Angular 16+ with TypeScript, reactive programming, and component-based architecture.



```
│  └──────────────────────────────┘                  │
└──────────────────────────────────────────────────────┘
```

Frontend Structure

```
frontend/
├── src/
│   ├── app/
│   │   ├── components/           # All UI components
│   │   │   ├── auth/             # Login/Signup
│   │   │   ├── dashboard/        # 3 role dashboards
│   │   │   ├── campaign/         # Campaign management
│   │   │   ├── sponsorship/      # Sponsorship requests
│   │   │   ├── payment/          # Payment processing
│   │   │   ├── notification/     # Notifications
│   │   │   ├── rating/           # Reviews/ratings
│   │   │   └── shared/           # Reusable components
│   │   │
│   │   ├── services/             # API communication
│   │   │   ├── auth.service.ts
│   │   │   ├── campaign.service.ts
│   │   │   ├── payment.service.ts
│   │   │   └── ...
│   │   │
│   │   ├── guards/               # Route protection
│   │   │   ├── auth.guard.ts     # Is user logged in?
│   │   │   └── role.guard.ts     # Role-based access
│   │   │
│   │   ├── interceptors/         # HTTP request/response
│   │   │   └── auth.interceptor.ts
│   │   │
│   │   ├── models/               # Data types
│   │   │   ├── user.model.ts
│   │   │   ├── campaign.model.ts
│   │   │   └── ...
│   │   │
│   │   ├── pipes/                # Data formatting
│   │   │   └── filter.pipe.ts
│   │   │
│   │   ├── app.module.ts         # Main module
│   │   ├── app-routing.module.ts # Routes
│   │   └── app.component.ts      # Root component
│   │
│   ├── environments/             # Configuration
│   │   ├── environment.ts        # Development
│   │   └── environment.prod.ts   # Production
│   │
│   ├── main.ts                   # Entry point
│   ├── index.html                # HTML template
```

```
│   └── styles.scss              # Global styles
│
├── package.json                 # Dependencies
├── angular.json                 # Angular config
└── tsconfig.json                # TypeScript config
```

2️⃣ App Module & Routing System

What is AppModule?

AppModule is the main module that bootstraps your Angular application. It declares all components, imports all dependencies (Material, Forms, HttpClient), and provides services to the entire app.

File: src/app/app.module.ts

Purpose:

- Declare all components
- Import Angular modules (Forms, HttpClient)
- Import 3rd party modules (Angular Material)
- Provide services globally
- Configure interceptors
- Setup dependency injection

AppModule Components Declaration

Component Category	Components Included	Purpose
Auth Components	LoginComponent, SignupComponent	User authentication (public)
Dashboard Components	AdminDashboard, BrandDashboard, InfluencerDashboard	Role-specific dashboards (protected)
Campaign Components	CampaignListComponent, CampaignFormComponent, CampaignDetailComponent	Campaign management
Domain Components	SponsorshipRequestComponent, PaymentComponent, RatingComponent, NotificationComponent	Feature modules

Shared Components	NavbarComponent, RatingDialogComponent, PaymentDialogComponent	Reusable across app

AppModule - Setup Code

app.module.ts - Complete Setup

```
@NgModule({ declarations: [ // All components must be declared here
AppComponent, LoginComponent, SignupComponent,
AdminDashboardComponent, BrandDashboardComponent,
InfluencerDashboardComponent, CampaignListComponent,
CampaignFormComponent, CampaignDetailComponent,
SponsorshipRequestComponent, PaymentComponent, RatingComponent,
NotificationComponent, NavbarComponent, RatingDialogComponent,
PaymentDialogComponent ], imports: [ // Angular core modules
BrowserModule, BrowserAnimationsModule, HttpClientModule,
FormsModule, ReactiveFormsModule, // Angular Material modules
MatToolbarModule, MatButtonModule, MatCardModule, MatInputModule,
MatFormFieldModule, MatSelectModule, MatTableModule,
MatPaginatorModule, MatSortModule, MatDialogModule,
MatSnackBarModule, // Routing AppRoutingModule ], providers: [ //
Interceptor registration { provide: HTTP_INTERCEPTORS, useClass:
AuthInterceptor, multi: true // Can have multiple interceptors }, //
Services are auto-provided with providedIn: 'root' ], bootstrap:
[AppComponent] // Root component }) export class AppModule { }
```

Routing System - Navigation

AppRoutingModule defines all routes (pages) in your application. Each route maps a URL path to a component. Routes can be protected with guards.

Path	Component	Guards	Public/Protected	Purpose
/	Redirects to /login	None	Public	Default route
/login	LoginComponent	None	Public	User login page
/signup	SignupComponent	None	Public	User registration

/dashboard/admin	AdminDashboardComponent	AuthGuard, RoleGuard	Protected (ADMIN)	Admin dashboard
/dashboard/brand	BrandDashboardComponent	AuthGuard, RoleGuard	Protected (BRAND)	Brand dashboard
/dashboard/influencer	InfluencerDashboardComponent	AuthGuard, RoleGuard	Protected (INFLUENCER)	Influencer dashboard
/campaigns	CampaignListComponent	AuthGuard	Protected	View all campaigns
/campaigns/new	CampaignFormComponent	AuthGuard, RoleGuard	Protected (BRAND)	Create new campaign
/campaigns/edit/:id	CampaignFormComponent	AuthGuard, RoleGuard	Protected (BRAND)	Edit campaign
/campaigns/:id	CampaignDetailComponent	AuthGuard	Protected	View campaign details
/sponsorship-requests	SponsorshipRequestComponent	AuthGuard	Protected	Sponsorship management
/payments	PaymentComponent	AuthGuard	Protected	Payment history
/ratings	RatingComponent	AuthGuard	Protected	View ratings
/notifications	NotificationComponent	AuthGuard	Protected	View notifications
**	Redirects to /login	None	-	Catch all (404)

3 Models & Data Types

What are Models?

Models (or Interfaces) define the shape of your data. They ensure type safety and make development easier with autocomplete.

Path: src/app/models/

Technology: TypeScript interfaces

Purpose: Type definition + API contract

Model Classes

Model File	Interfaces Defined	Purpose
user.model.ts	User, AuthResponse, LoginRequest, RegisterRequest, ChangePasswordRequest	User authentication & profile
campaign.model.ts	Campaign, CampaignRequest, CampaignFilter	Campaign data structure
sponsorship.model.ts	SponsorshipRequest, WorkSubmission, SponsorshipApplicationRequest	Sponsorship/application data
payment.model.ts	Payment, PaymentRequest, PaymentStatus	Payment data
notification.model.ts	Notification, NotificationStatus	Notification data
rating.model.ts	Rating, RatingRequest	Review & rating data
common.model.ts	ApiResponse, Pagination, Sort	Common/shared structures

Example Models

user.model.ts - User Interfaces

```
export interface User { id: number; name: string; email: string; role: 'ADMIN' | 'BRAND' | 'INFLUENCER'; bio?: string; // Optional profileImage?: string; // Optional } export interface AuthResponse { token: string; // JWT token from backend type: string; // "Bearer" id: number; name: string; email: string; role: string; } export interface LoginRequest { email: string; password: string; } export interface RegisterRequest { name: string; email: string; password: string; role: 'ADMIN' | 'BRAND' | 'INFLUENCER'; }
```

campaign.model.ts - Campaign Interfaces

```
export interface Campaign { id: number; name: string; description: string; platform: string; // Instagram, Twitter, etc budget: number; startDate: Date; endDate: Date; status: 'ACTIVE' | 'PAUSED' | 'COMPLETED' | 'CANCELLED' | 'EXPIRED'; brandId: number; // Campaign creator createdAt: Date; updatedAt: Date; } export interface CampaignRequest { name: string; description: string; platform: string; budget: number; startDate: Date; endDate: Date; }
```

4️⃣ Services Layer - API Communication

What is a Service?

Angular Services contain reusable logic and communicate with the backend API. Each service corresponds to one domain (Auth, Campaign, Payment, etc).

Path: src/app/services/

Pattern: @Injectable, HttpClient

Provides: Methods to call backend endpoints

Services Overview

Service	Backend Domain	Key Methods	Called By
AuthService	/api/auth	register(), login(), logout(), getCurrentUser()	Auth components
CampaignService	/api/campaigns	getCampaigns(), getCampaignById(), createCampaign(), updateCampaign()	Campaign components
SponsorshipService	/api/influencer	applyToCampaign(), submitWork(), getMyApplications()	Sponsorship components
PaymentService	/api/brand	processPayment(), getPaymentHistory(), getPaymentStatus()	Payment components

NotificationService	/api/notifications	getNotifications(), markAsRead(), clearNotifications()	Navbar component
RatingService	/api/campaigns/rate	submitRating(), getRatings(), getAverageRating()	Rating components
AdminService	/api/admin	getDashboardStats(), getAllUsers(), suspendUser()	Admin dashboard

Example: AuthService - Complete Implementation

auth.service.ts - Authentication Logic

```
@Injectable({ providedIn: 'root' }) export class AuthService { private apiUrl = `${environment.apiUrl}/auth`; private currentUserSubject = new BehaviorSubject<User | null>(null); public currentUser$ = this.currentUserSubject.asObservable(); constructor(private http: HttpClient, private router: Router) { // Load user from localStorage on service init this.loadUserFromStorage(); } // ===== REGISTER ===== register(request: RegisterRequest): Observable<AuthResponse> { return this.http.post<AuthResponse>(`${this.apiUrl}/register`, request) .pipe( tap(response => this.handleAuthResponse(response)) ); } // ===== LOGIN ===== login(request: LoginRequest): Observable<AuthResponse> { return this.http.post<AuthResponse>(`${this.apiUrl}/login`, request) .pipe( tap(response => this.handleAuthResponse(response)) ); } // ===== HANDLE AUTH RESPONSE (After login/register) ===== private handleAuthResponse(response: AuthResponse): void { // Save token to localStorage localStorage.setItem('token', response.token); // Create User object (without password) const user: User = { id: response.id, name: response.name, email: response.email, role: response.role as 'ADMIN' | 'BRAND' | 'INFLUENCER' }; // Save user to localStorage localStorage.setItem('user', JSON.stringify(user)); // Notify all subscribers with new user this.currentUserSubject.next(user); } // =====
```

```
GET CURRENT USER ===== getCurrentUser(): User | null {
return this.currentUserSubject.value; } // ===== IS
LOGGED IN? ===== isLoggedIn(): boolean { return
!!this.getToken(); } // ===== GET TOKEN =====
getToken(): string | null { return
localStorage.getItem('token'); } // ===== LOGOUT =====
logout(): void { // Clear localStorage
localStorage.removeItem('token');
localStorage.removeItem('user'); // Reset current user
this.currentUserSubject.next(null); // Redirect to login
this.router.navigate(['/login']); } // ===== LOAD USER
FROM STORAGE (Refresh page) ===== private
loadUserFromStorage(): void { const userData =
localStorage.getItem('user'); if (userData) { // User
was logged in, restore from storage
this.currentUserSubject.next(JSON.parse(userData)); } }
} // Usage in components: // constructor(private
authService: AuthService) { } // // onLogin() { //
this.authService.login(loginRequest).subscribe( //
response => { console.log('Logged in:', response); }, //
error => { console.log('Login failed:', error); } // );
// } // // currentUser = this.authService.currentUser$;
// Observable!
```

5️⃣ Guards - Route Protection

What are Guards?

Guards are functions that protect routes. They check if a user should be allowed to access a page before route activation.

Path: src/app/guards/

Types:
• CanActivate - Can user access this route?
• CanDeactivate - Should user leave this route?
• Resolve - Load data before showing component?

Your App Uses: CanActivate guards only

Guard Classes

Guard	Purpose	Check	Used On Routes
AuthGuard	Require login	Is user logged in? (has token?)	Most protected routes
RoleGuard	Require specific role	Does user have required role? (ADMIN, BRAND, INFLUENCER)	Dashboards, admin pages

AuthGuard - Login Check

auth.guard.ts - Check if User Logged In

```
@Injectable({ providedIn: 'root' }) export class AuthGuard implements CanActivate { constructor(private authService: AuthService, private router: Router) {} // ===== CAN ACTIVATE METHOD ===== canActivate(): boolean { // Check if user is logged in if (this.authService.isLoggedIn()) { return true; // User logged in → Allow access } // User not logged in → Redirect to login this.router.navigate(['/login']); return false; // Block access } } // Usage in routing: // { // path: 'dashboard/brand', // component: BrandDashboardComponent, // canActivate: [AuthGuard] // Protect with AuthGuard // } // Flow: // User clicks on protected route // → AuthGuard.canActivate() called // → Is user logged in? (has token in localStorage?) // → YES: Navigate to page // → NO: Redirect to /login
```

RoleGuard - Role-Based Access

role.guard.ts - Check User Role

```
@Injectable({ providedIn: 'root' }) export class RoleGuard implements CanActivate { constructor(private authService: AuthService, private router: Router) {} canActivate(route: ActivatedRouteSnapshot): boolean { // Get required role from route metadata const requiredRole = route.data['role']; // From route config // If no role requirement, allow if (!requiredRole) { return true; } // Get current user const currentUser = this.authService.getCurrentUser(); // User not logged in if (!currentUser) { this.router.navigate(['/login']); return false; } // Check if user has required role if (currentUser.role === requiredRole) { return true; // User has correct role } // User doesn't have required role this.router.navigate(['/unauthorized']); return false; } } // Usage in routing: // { // path: 'dashboard/admin', // component: AdminDashboardComponent, // canActivate: [AuthGuard, RoleGuard], // data: { role: 'ADMIN' } // Required role in data // } // Flow: // User clicks /dashboard/admin // → AuthGuard: Is logged
```

```
in? YES // → RoleGuard: Has role ADMIN? // → YES:
Show admin dashboard // → NO: Redirect to
/unauthorized
```

6️⃣ Interceptors - Request/Response Processing

What is an Interceptor?

Interceptors are middleware that process every HTTP request and response. They run automatically before sending request and after receiving response.

Path: src/app/interceptors/

Usage:

- Add JWT token to every request
- Handle 401 errors (token expired)
- Log requests/responses
- Add headers automatically

AuthInterceptor - JWT Token Management

auth.interceptor.ts - Automatic Token Addition

```
@Injectable() export class AuthInterceptor
implements HttpInterceptor {
constructor(private authService:
AuthService, private router: Router) {}
intercept(request: HttpRequest<any>, next:
HttpHandler): Observable<HttpEvent<any>> {
// ===== ADD TOKEN TO REQUEST ===== const
token = this.authService.getToken(); if
(token) { // Clone request and add
Authorization header request =
request.clone({ setHeaders: { Authorization:
`Bearer ${token}` // Header becomes:
Authorization: Bearer eyJhbGciOiJ... } }); }
// ===== SEND REQUEST ===== return
next.handle(request).pipe( // ===== HANDLE
```

```
RESPONSE/ERRORS ===== catchError((error:
HttpErrorResponse) => { // If 401 (token
expired or invalid) if (error.status ===
401) { // Clear token
this.authService.logout(); // Redirect to
login this.router.navigate(['/login']); } //
Pass error to component return throwError(()
=> error); }) ); } } // ===== FLOW ===== //
1. Frontend makes HTTP request // 2.
AuthInterceptor.intercept() called // 3. Get
token from localStorage // 4. Add token to
Authorization header // 5. Send request to
backend with token header // 6. Backend
validates token // 7. Receive response // 8.
If 401: logout user and redirect // 9.
Return response to component // =====
EXAMPLE REQUEST ===== // Before Interceptor:
// GET /api/campaigns // Headers: { Content-
Type: application/json } // // After
Interceptor: // GET /api/campaigns //
Headers: { // Content-Type:
application/json, // Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... // }
```

7️⃣ Auth Components - Login & Signup

Authentication Flow in Frontend

```
User visits app
    ↓
Frontend loads AppComponent
    ↓
Browser checks localStorage for token
    ↓
Token exists?
    ├─ YES: User logged in, redirect to dashboard
    └─ NO: Show login page
    ↓
User sees Login & Signup pages
```

Auth Components

Component	Purpose	On Submit	On Success
LoginComponent	User login	Call authService.login()	Save token, redirect to dashboard
SignupComponent	User registration	Call authService.register()	Save token, redirect to dashboard

LoginComponent - User Login

login.component.ts - Login Functionality

```
@Component({ selector: 'app-login',
templateUrl: './login.component.html',
styleUrls: ['./login.component.scss'] })
export class LoginComponent implements
OnInit { loginForm: FormGroup; loading =
false; submitted = false; constructor(
private formBuilder: FormBuilder, private
authService: AuthService, private router:
Router ) { // Initialize form with
validation this.loginForm =
this.formBuilder.group({ email: ['',
[Validators.required, Validators.email]],
password: ['', [Validators.required,
Validators.minLength(6)]] }); } ngOnInit():
void { // If already logged in, redirect to
dashboard if (this.authService.isLoggedIn())
{
this.router.navigate(['/dashboard/brand']);
// or admin/influencer } } get f() { return
this.loginForm.controls; // For template
shorthand } onSubmit(): void {
this.submitted = true; // If form invalid,
stop if (this.loginForm.invalid) { return; }
this.loading = true; // Call backend
this.authService.login(this.loginForm.value).su
response => { // Login successful!
console.log('Login successful:', response);
// Token already saved in
authService.login() // Navigate to dashboard
based on role if (response.role === 'ADMIN')
{
this.router.navigate(['/dashboard/admin']);
} else if (response.role === 'BRAND') {
this.router.navigate(['/dashboard/brand']);
} else {
this.router.navigate(['/dashboard/influencer'])
} }, error => { // Login failed
console.log('Login failed:', error);
this.loading = false; } ); } }
```

8 Dashboard Components - Three User Roles

Role-Based Dashboards

Each user role (ADMIN, BRAND, INFLUENCER) has a different dashboard showing role-specific data and functionality.

Dashboard	Role	Shows	Functions
AdminDashboard	ADMIN	Platform stats, users, campaigns, payments	View all users, suspend accounts, statistics
BrandDashboard	BRAND	My campaigns, active sponsorships, earnings	Create campaign, manage applications, see payments
InfluencerDashboard	INFLUENCER	Available campaigns, my applications, earnings	Browse campaigns, apply, complete work, get paid

AdminDashboardComponent - Platform Overview

admin-dashboard.component.ts - Admin Stats

```
@Component({ selector: 'app-admin-
dashboard', templateUrl: './admin-
dashboard.component.html' }) export class
AdminDashboardComponent implements OnInit {
dashboardStats: DashboardStats; allUsers:
User[] = []; loading = true; constructor(
private adminService: AdminService, private
authService: AuthService ) {
this.dashboardStats = {} as DashboardStats;
} ngOnInit(): void { // Check if current
user is ADMIN const currentUser =
this.authService.getCurrentUser(); if
(currentUser?.role !== 'ADMIN') { return; //
Should not reach here due to RoleGuard } //
Load statistics
this.adminService.getDashboardStats().subscribe
stats => { this.dashboardStats = stats; //
Shows: totalUsers, totalCampaigns,
totalEarnings, etc this.loading = false; },
error => { console.log('Failed to load
stats:', error); this.loading = false; } );
// Load all users list
this.adminService.getAllUsers().subscribe(
users => { this.allUsers = users; } ); } //
Suspend user account suspendUser(userId:
number): void {
this.adminService.suspendUser(userId).subscribe
() => { console.log('User suspended'); //
Reload users list this.ngOnInit(); } ); } }
```

BrandDashboardComponent - Brand Overview

brand-dashboard.component.ts - Brand Stats

```
@Component({ selector: 'app-brand-
dashboard', templateUrl: './brand-
dashboard.component.html' }) export class
BrandDashboardComponent implements OnInit {
campaigns: Campaign[] = [];
recentApplications: SponsorshipRequest[] =
[]; earnings = 0; loading = true;
constructor( private campaignService:
CampaignService, private sponsorshipService:
```

```
SponsorshipService, private authService:
AuthService ) {} ngOnInit(): void { // Get
current brand user const currentUser =
this.authService.getCurrentUser(); // Load
brand's campaigns
this.campaignService.getMyBrandCampaigns().subs
campaigns => { this.campaigns = campaigns;
// Shows: active campaigns, total
applications, earnings this.loading = false;
} ); // Load recent applications
this.sponsorshipService.getApplicationsToBrandC
applications => { this.recentApplications =
applications; } ); } // Approve influencer
application
approveApplication(applicationId: number):
void {
this.sponsorshipService.approveApplication(appl
() => { // Notification sent to influencer
this.ngOnInit(); // Reload } ); } // Create
new campaign createNewCampaign(): void { //
Navigate to campaign form } }
```

9 Campaign Components - List, Form, Detail

Campaign Management

Component	Purpose	URL	Features
CampaignListComponent	View all campaigns	/campaigns	Table, filter, sort, pagination, search
CampaignFormComponent	Create/edit campaign	/campaigns/new or /campaigns/edit/:id	Form with validation, upload, date picker
CampaignDetailComponent	View campaign details	/campaigns/:id	Full details, applications list, apply button

CampaignListComponent - View All Campaigns

campaign-list.component.ts - Campaign Listing

```
@Component({ selector: 'app-campaign-list', templateUrl: './campaign-list.component.html' }) export class CampaignListComponent implements OnInit, OnDestroy { campaigns: Campaign[] = []; filteredCampaigns: Campaign[] = []; loading = false; // Table properties displayedColumns: string[] = ['name', 'platform', 'budget', 'status', 'actions'];
```

```
paginator: MatPaginator; sort: MatSort; //
Filters searchText = ''; selectedStatus:
string = ''; selectedPlatform: string = '';
private destroy$ = new Subject<void>();
constructor( private campaignService:
CampaignService, private dialog: MatDialog )
{} ngOnInit(): void { this.loadCampaigns();
} // ===== LOAD CAMPAIGNS =====
loadCampaigns(): void { this.loading = true;
this.campaignService.getAllCampaigns().pipe(
takeUntil(this.destroy$) ).subscribe(
campaigns => { this.campaigns = campaigns;
this.applyFilters(); this.loading = false;
}, error => { console.log('Failed to load
campaigns:', error); this.loading = false; }
); } // ===== APPLY FILTERS =====
applyFilters(): void {
this.filteredCampaigns =
this.campaigns.filter(campaign => { // Text
search const matchesSearch =
campaign.name.toLowerCase()
.includes(this.searchText.toLowerCase()); //
Status filter const matchesStatus =
!this.selectedStatus || campaign.status ===
this.selectedStatus; // Platform filter
const matchesPlatform =
!this.selectedPlatform || campaign.platform
=== this.selectedPlatform; return
matchesSearch && matchesStatus &&
matchesPlatform; }); } // ===== VIEW DETAILS
===== viewCampaign(campaignId: number): void
{ // Navigate to /campaigns/{campaignId} }
ngOnDestroy(): void { this.destroy$.next();
this.destroy$.complete(); } }
```

🔟 Domain Components - Sponsorship, Payment, Rating

Domain Feature Components

Component	Feature	Purpose	Users
SponsorshipRequestComponent	Influencer Applications	Apply to campaigns, track applications	Influencer, Brand
PaymentComponent	Payment Processing	Process payments, view history	Brand, Influencer
RatingComponent	Reviews & Ratings	Submit and view ratings	All users
NotificationComponent	Notifications	View and manage notifications	All users

SponsorshipRequestComponent - Apply to Campaign

sponsorship-request.component.ts - Influencer Applications

```
@Component({ selector: 'app-sponsorship-request', templateUrl: './sponsorship-request.component.html' }) export class SponsorshipRequestComponent implements
```

```
OnInit { myApplications:
SponsorshipRequest[] = [];
acceptedApplications: SponsorshipRequest[] =
[]; loading = false; constructor( private
sponsorshipService: SponsorshipService,
private authService: AuthService ) {}
ngOnInit(): void { const currentUser =
this.authService.getCurrentUser(); if
(currentUser?.role === 'INFLUENCER') { //
Load influencer's applications
this.loadMyApplications(); } else if
(currentUser?.role === 'BRAND') { // Load
brand's incoming applications
this.loadBrandApplications(); } } // =====
LOAD MY APPLICATIONS (Influencer) =====
loadMyApplications(): void {
this.sponsorshipService.getMyApplications().suk
applications => { this.myApplications =
applications; // Shows pending, accepted,
rejected applications } ); } // ===== APPLY
TO CAMPAIGN =====
applyToCampaign(campaignId: number,
proposal: string): void { const request = {
campaignId: campaignId, proposal: proposal
};
this.sponsorshipService.applyToCampaign(request
response => { alert('Application
submitted!'); this.loadMyApplications(); //
Reload }, error => { alert('Failed to apply:
' + error.message); } ); } // ===== SUBMIT
WORK (After approval) =====
submitWork(applicationId: number,
workDescription: string): void { const
request = { sponsorshipId: applicationId,
workDescription: workDescription };
this.sponsorshipService.submitWork(request).suk
response => { alert('Work submitted
successfully!'); this.loadMyApplications();
} ); } }
```

1️⃣1️⃣ Shared Components - Reusable UI

Reusable Components

Component	Purpose	Used In	Type
NavbarComponent	Header with logo, navigation, notifications	App root (on every page)	Fixed header
RatingDialogComponent	Modal to submit rating/review	Campaign detail, after completion	Dialog (popup)
PaymentDialogComponent	Modal to process payment	Payment page, sponsorship approval	Dialog (popup)

NavbarComponent - Header Navigation

navbar.component.ts - Top Navigation

```
@Component({ selector: 'app-navbar',
templateUrl: './navbar.component.html' })
export class NavbarComponent implements
OnInit { currentUser$: Observable<User |
null>; notifications: Notification[] = [];
unreadCount = 0; constructor( private
authService: AuthService, private
notificationService: NotificationService,
private router: Router ) { // Subscribe to
current user this.currentUser$ =
this.authService.currentUser$; } ngOnInit():
void { // Load notifications
```

```
this.loadNotifications(); // Refresh
notifications every 30 seconds
setInterval(() => this.loadNotifications(),
30000); } // ===== LOAD NOTIFICATIONS =====
loadNotifications(): void {
this.notificationService.getNotifications().sub
notifications => { this.notifications =
notifications; this.unreadCount =
notifications.filter(n => !n.isRead).length;
} ); } // ===== LOGOUT ===== logout(): void
{ this.authService.logout(); // logout()
handles redirect to /login } // ===== MARK
AS READ =====
markNotificationAsRead(notificationId:
number): void {
this.notificationService.markAsRead(notificatic
() => { this.loadNotifications(); } ); } //
===== NAVIGATE BY ROLE =====
navigateToDashboard(): void { const user =
this.authService.getCurrentUser(); if
(user?.role === 'ADMIN') {
this.router.navigate(['/dashboard/admin']);
} else if (user?.role === 'BRAND') {
this.router.navigate(['/dashboard/brand']);
} else {
this.router.navigate(['/dashboard/influencer'])
} } }
```

1️⃣2️⃣ Pipes - Data Formatting

What are Pipes?

Pipes transform data for display. They format data in templates without modifying the source.

Pipe Files

Pipe	Purpose	Example
FilterPipe	Filter array items	{{ campaigns | filter: searchText }}
DatePipe (built-in)	Format dates	{{ campaign.startDate | date: 'shortDate' }}
CurrencyPipe (built-in)	Format currency	{{ campaign.budget | currency }}
UpperCasePipe (built-in)	Convert to uppercase	{{ user.role | uppercase }}

FilterPipe - Custom Pipe

filter.pipe.ts - Array Filtering

```
@Pipe({ name: 'filter' }) export class FilterPipe implements PipeTransform { transform(items: Campaign[], searchText: string): Campaign[] { if (!items || !searchText) { return items; // Return as-is if no filter } // Filter items matching search text return items.filter((item: Campaign) =>
```

```
item.name.toLowerCase().includes(searchText.toI
||
item.platform.toLowerCase().includes(searchText
); } } // Usage in template: // <mat-card
*ngFor="let campaign of (campaigns | filter:
searchText)"> // Example: // campaigns = [
// { name: 'Summer Fashion', platform:
'Instagram' }, // { name: 'Beach Vibes',
platform: 'Twitter' }, // { name: 'Summer
Sales', platform: 'TikTok' } // ] // //
searchText = 'Summer' // // Result after
pipe: // campaigns | filter: 'Summer'
returns: // [ // { name: 'Summer Fashion',
platform: 'Instagram' }, // { name: 'Summer
Sales', platform: 'TikTok' } // ]
```

1️⃣3️⃣ Angular Material Integration

What is Angular Material?

Angular Material is a library of pre-built UI components (buttons, cards, tables, dialogs) following Google's Material Design guidelines.

Material Components Used

Component	Purpose	Usage	Example
MatToolbar	Header bar	Navbar	<mat-toolbar>
MatButton	Clickable button	All pages	<button mat-button>
MatCard	Content container	Campaign cards, forms	<mat-card>
MatTable	Data table with sorting	Campaign list, users list	<table mat-table>
MatDialog	Modal popup	Rating, payment dialogs	dialog.open(Component)
MatForm	Form inputs	Login, signup, forms	<mat-form-field>
MatSnackBar	Toast notifications	Success/error messages	snackBar.open('Message')

1️⃣4️⃣ Complete User Flows

Flow 1: User Registration & Login

```
1. User visits app → Redirects to /login

2. Click SIGNUP button → Navigate to /signup

3. SignupComponent shows form:
   ├─ Name input
   ├─ Email input
   ├─ Password input
   ├─ Role dropdown (ADMIN, BRAND, INFLUENCER)
   └─ REGISTER button

4. User fills form & clicks REGISTER

5. SignupComponent validates:
   ├─ Email format valid? ✓
   ├─ Password min 6 chars? ✓
   └─ All fields filled? ✓

6. RegisterRequest sent to backend:
   POST /api/auth/register
   {
     "name": "John Doe",
     "email": "john@example.com",
     "password": "password123",
     "role": "INFLUENCER"
   }

7. AuthInterceptor adds JWT token header:
   Authorization: Bearer ...

8. Backend validates & creates user

9. Backend returns AuthResponse:
   {
     "token": "eyJhbGciOiJ...",
     "id": 1,
     "name": "John Doe",
     "email": "john@example.com",
     "role": "INFLUENCER"
   }

10. AuthService.register() handles response:
    ├─ Save token to localStorage
```

```
    ├─ Save user to localStorage
    └─ Update currentUserSubject

11. SignupComponent checks role:
    ├─ ADMIN → Redirect to /dashboard/admin
    ├─ BRAND → Redirect to /dashboard/brand
    └─ INFLUENCER → Redirect to /dashboard/influencer

12. Dashboard component loads
    ├─ AuthGuard checks: is logged in? ✓
    ├─ RoleGuard checks: has correct role? ✓
    └─ Loads dashboard data

✓ USER IS NOW LOGGED IN AND ON DASHBOARD!
```

Flow 2: Influencer Applies to Campaign

```
1. INFLUENCER user logged in on /dashboard/influencer

2. Click "Browse Campaigns" → Navigate to /campaigns

3. CampaignListComponent loads:
   ├─ Call campaignService.getAllCampaigns()
   ├─ AuthInterceptor adds token
   ├─ Display campaigns in table

4. Click campaign → Navigate to /campaigns/{campaignId}

5. CampaignDetailComponent shows:
   ├─ Campaign name, description, budget
   ├─ Start/end dates
   ├─ Current applications count
   └─ "APPLY FOR THIS CAMPAIGN" button

6. Influencer fills proposal & clicks APPLY

7. ApplyRequest sent:
   POST /api/influencer/apply
   {
     "campaignId": 5,
     "proposal": "I have great Instagram following..."
   }

8. Backend creates SponsorshipRequest in PENDING status

9. Backend sends notification to Brand:
   "New application from John Doe"
```

10. Frontend shows success message

11. Influencer sees application in:
    /sponsorship-requests → Status: PENDING

12. BRAND receives notification in navbar

13. Brand clicks notification → /sponsorship-requests

14. BrandDashboardComponent shows:
    ├─ List of applications to brand's campaigns
    ├─ Influencer name, proposal
    └─ APPROVE / REJECT buttons

15. Brand clicks APPROVE

16. ApprovalRequest sent:
    POST /api/brand/approve-application/{applicationId}

17. Backend changes status to ACCEPTED

18. Backend sends notification:
    "Your application was approved!"

19. Influencer receives notification

20. Influencer must now SUBMIT WORK

21. Influencer navigates to /sponsorship-requests

22. SponsorshipRequestComponent shows ACCEPTED application

23. Click "SUBMIT WORK"

24. Modal opens for work submission:
    ├─ Text area for description
    ├─ File upload for proof
    └─ SUBMIT button

25. WorkSubmissionRequest sent:
    POST /api/influencer/submit-work
    {
      "sponsorshipId": 12,
      "workDescription": "Posted Instagram story...",
      "proofFile": [binary]
    }

26. Backend checks work & approves

27. Backend creates Payment record

28. Notification sent: "Payment ready!"

29. Influencer navigates to /payments

30. PaymentComponent shows:
    ├─ Campaign name
    ├─ Amount: $500
    ├─ Status: PENDING
    └─ View proof button

✓ PAYMENT PROCESS COMPLETE FOR THIS CAMPAIGN!

📚 Complete Frontend Architecture Summary

Frontend Stack

⚙️ Framework

Angular 16+
TypeScript-based framework
Component-based architecture

🎨 UI Library

Angular Material
Pre-built components
Material Design spec

🔗 HTTP Client

HttpClientModule
Communicates with backend
JSON serialization

📝 Forms

Reactive Forms
FormBuilder, FormGroup
Validation

🛡️ Authentication

JWT Tokens
localStorage storage
Interceptor injection

🎯 Routing

Guards + Routes
Protected routes
Lazy loading ready

Component Folder Structure

```
Components (8 domains)
├── auth/
│   ├── login/
│   │   ├── login.component.ts
│   │   ├── login.component.html
│   │   └── login.component.scss
```

```
│   └── signup/
│       ├── signup.component.ts
│       ├── signup.component.html
│       └── signup.component.scss
│
├── dashboard/
│   ├── admin-dashboard/
│   ├── brand-dashboard/
│   └── influencer-dashboard/
│
├── campaign/
│   ├── campaign-list/
│   ├── campaign-form/
│   └── campaign-detail/
│
├── sponsorship/
│   └── sponsorship-request/
│
├── payment/
│   └── payment.component.ts
│
├── notification/
│   └── notification.component.ts
│
├── rating/
│   └── rating.component.ts
│
└── shared/
    ├── navbar/
    ├── rating-dialog/
    └── payment-dialog/
```

Data Flow Architecture

```
User Interaction (Click Button)
    ↓
Component Event Handler
    ↓
Call Service Method
    ↓
Service Creates HTTP Request
    ↓
AuthInterceptor adds JWT Token
    ↓
HTTP Request sent to Backend
    ↓
Backend processes request
    ↓
Backend sends response
```

```
    ↓
AuthInterceptor checks status
    ↓ (If 401)
Logout user & redirect to /login
    ↓ (If 200)
Return response to service
    ↓
Service transforms data
    ↓
Component receives DataComponent updates template
    ↓
Browser renders new UI
    ↓
User sees result
```

✅ You Now Understand Frontend Architecture!

You know:

✓ All 8 component domains

✓ All 7 services and what they do

✓ Guards for route protection

✓ Interceptor for JWT injection

✓ Models for data types

✓ Routing system

✓ Angular Material components

✓ Complete user flows

✓ How frontend communicates with backend

✓ Authentication and authorization system